AI Architecture

Codebase Explorer — A proactive, adaptive codebase understanding system

7-Stage Pipeline	Hybrid RAG	Proactive Engine	Real-time Streaming
Claude Sonnet + Haiku	pgvector + Full-text	Deterministic Rules	SSE via Hono

React + Vite + Zustand + Canvas | Node.js + Hono | Supabase + pgvector | Anthropic SDK + OpenAI Embeddings

Contents

1	**Design Principles** Core architectural philosophy
2	**System Architecture** Three-tier overview with visual diagram
3	**Ingestion Pipeline** 7-stage ZIP-to-knowledge-base transformation
4	**Semantic Memory (RAG)** Hybrid retrieval with pgvector + full-text search
5	**User Model** Exploration tracking and understanding estimation
6	**Proactive Engine** Deterministic UI guidance system
7	**Chat System** RAG-powered streaming conversation
8	**End-to-End Data Flows** Upload, explore, and chat workflows
9	**File Structure** Server and client directory layout
10	**Cost Estimates** Per-pipeline and per-interaction costs

1. Design Principles

1

No frameworks
Uses Anthropic's SDK directly with composable patterns (prompt chaining, parallelization, orchestrator-workers, routing, evaluator-optimizer). No LangChain, no LangGraph.

2

Structured outputs everywhere
Every Claude call uses tool-use (tool_choice: { type: 'tool', name: schemaName }) for guaranteed valid JSON. No regex parsing, no try/catch on malformed responses.

3

Stream everything user-facing
Pipeline progress, chat responses, and proactive insights all stream via SSE. The user never stares at a spinner wondering if something is broken.

4

Start simple, add complexity only where it improves outcomes
Each pipeline stage exists because a single call demonstrably cannot produce the same quality output.

5

Grounded in real code
Every explanation, insight, and answer references actual files and line numbers from the user's codebase, retrieved via RAG — never from Claude's general training knowledge.

2. System Architecture

CLIENT (React + Vite)

```
Upload → Processing UI → Explorer (Graph + Inspector + Chat)
Zustand store ← SSE streams from backend
```

HTTPS / SSE

API SERVER (Node.js + Hono, port 3007)

```
POST /api/pipeline/start      → Kick off ingestion pipeline
GET  /api/pipeline/:id/stream  → SSE pipeline progress
POST /api/chat                 → RAG-powered chat (streaming)
POST /api/proactive            → Get next proactive action
POST /api/explain              → Explain a specific node

Anthropic SDK (Sonnet + Haiku)  |  OpenAI Embeddings  |  Supabase
```

SUPABASE

```
Tables: projects, files, concepts, concept_edges,
        code_chunks, user_state, insights, chat_messages

pgvector: HNSW index (cosine)  |  Full-text: GIN index
Hybrid search: 70% vector / 30% lexical
```

Three-tier architecture: React client, Hono API server, and Supabase persistence layer.

Why Hono

The app is Vite + React. Adding Next.js would mean migrating the entire frontend or running two separate frameworks. Cloudflare Workers have execution time limits that complicate multi-minute pipelines. **Hono** is lightweight (~14KB), runs on Node.js alongside the existing Vite dev server, has first-class SSE streaming support, and can deploy to Cloudflare Workers, Vercel Edge, or plain Node.js later. In development, Vite proxies /api/* to the Hono server on port 3007.

3. Ingestion Pipeline

The pipeline transforms a ZIP file into a rich, queryable knowledge base. It runs as an async background job — the client gets a project ID immediately and subscribes to progress updates via SSE from /api/pipeline/:id/stream.

ZIP Upload: Client-side

User drops file

1 Stage 1: File Extraction

Parse ZIP, detect imports, classify files

2 Stage 2: File Analysis

Batches of 15 → Claude Haiku (sequential, 60s waits)

3 Stage 3: Concept Synthesis

1 Sonnet call → concept graph (3-20 nodes)

★ *Graph renders here — user can start exploring*

4 Stage 4: Depth Mapping

1 Sonnet call → beginner/intermediate/advanced explanations

5 Stage 5: Insight Generation

1 Sonnet call → 10-20 insights (6 categories)

6 Stage 6: Embedding & Indexing

~800-token chunks → OpenAI embeddings → pgvector

7 Stage 7: Proactive Seeding

1 Sonnet call → exploration path for user

7-stage pipeline from ZIP upload to fully-indexed knowledge base.

Stage 1: File Extraction & Classification

Runs on: Client (fileParser.js)

- Extracts ZIP, filters out non-code files (node_modules, .git, dist, build)
- Identifies code files by extension
- Extracts imports via regex (ES6, CommonJS, Python, Go, Rust)
- Resolves relative import paths to actual file paths
- Sends file tree, contents, and import edges to server

Stage 2: Sequential File Analysis

Runs on: Server → Claude Haiku

- Files processed in batches of 15 (content truncated to 1,500 chars)
- Batches run sequentially with 60-second waits between (rate limiting)
- Structured output via tool-use: path, purpose, concepts, key_exports, depends_on, complexity, role
- Failed batches fall back to minimal stub analyses
- Results stored in Supabase files table

Stage 3: Concept Synthesis

Runs on: Server → 1 Claude Sonnet call

- Receives all file analyses + framework detection
- Identifies natural conceptual groupings (3-20 concepts)
- Each concept: id, name, emoji, color, metaphor, one_liner, explanation, deep_explanation
- Generates edges with relationship labels and strength
- Returns suggested starting concept and codebase summary
- Graph renders in UI at this point — user can start exploring

Stage 4: Depth Mapping

Runs on: Server → 1 Claude Sonnet call

- Generates multi-level explanations for each concept
- Beginner: only analogies and everyday language, no code terms
- Intermediate: mentions technical terms but explains inline
- Advanced: assumes programming familiarity, references specific patterns
- Powers the adaptive explanation system in InspectorPanel

Stage 5: Insight Generation

Runs on: Server → 1 Claude Sonnet call

- Prompt: think like a senior engineer doing a code review
- 6 categories: architecture, risk, pattern, praise, suggestion, complexity
- Each insight: title, category, summary, detail, related concepts/files
- Priority scoring (1-10) and prerequisite concepts for ordering

Stage 6: Embedding & Indexing

Runs on: Server → OpenAI API

- Splits each file into ~800-token chunks with 10% overlap
- Contextual prefix per chunk: file path, purpose, concept name
- Embeds with OpenAI text-embedding-3-small (1536 dimensions)
- Batch processing: up to 2,048 texts per API call
- Stored in pgvector with HNSW index (cosine) + GIN full-text index

Stage 7: Proactive Seeding

Runs on: Server → 1 Claude Sonnet call

- Generates ordered exploration path for the user
- Based on concept graph, insights, and suggested starting concept
- Not a rigid tour — proactive engine adjusts in real-time
- Stored in user_state table as exploration_path array

4. Semantic Memory (RAG)

Every time the app explains something, it must reference actual code. The RAG system ensures explanations are grounded in the user's codebase, not Claude's training data.

Retrieval Flow

1. Generate query embedding from the question/topic (OpenAI text-embedding-3-small)
2. Call search_code_chunks RPC via Supabase with query embedding + optional concept filter
3. Hybrid search: 70% vector similarity (cosine) + 30% lexical (tsvector)
4. Fallback: if RPC fails, simple full-text search without embeddings
5. Format retrieved chunks as XML-tagged context for Claude
6. Stream Claude response with retrieved context + user's understanding level

Database Schema

Table	Purpose	Key Columns
projects	One row per uploaded codebase	name, framework, pipeline_status, pipeline_progress
files	Every code file with analysis	path, content, analysis (jsonb), concept_id, role
concepts	Extracted concepts	name, emoji, metaphor, explanation, beginner/intermediate/advanced
concept_edges	Relationships between concepts	source_key, target_key, relationship, strength
code_chunks	Chunked code for RAG	content, context_summary, embedding (vector 1536), fts
user_state	Exploration tracking	explored_concepts, time_per_concept, understanding_level
insights	Proactive insights queue	title, category, summary, priority, requires_understanding
chat_messages	Conversation history	role, content, context (jsonb)

5. User Model

Tracking Signals

Signal	How Collected	What It Tells Us
Concepts viewed	Click/tap on concept node	What they've seen
Time per concept	Timer from select to deselect	How much they engaged
Files opened	Click on file in inspector	Depth of exploration
Insights interaction	Dismiss vs expand behavior	What interests them
Questions asked	Chat input	What they don't understand

Understanding Level Estimation

Level	Time Threshold	UI Behavior
unseen	Not viewed	Default explanation hidden
glanced	< 5 seconds	Brief summary shown
beginner	5 - 30 seconds	Beginner-level explanation (analogies only)
intermediate	30 - 120 seconds	Intermediate explanation (technical terms explained)
advanced	> 120 seconds	Advanced explanation (patterns & libraries)

User state updates are **debounced and sent to Supabase every 5 seconds** via PATCH /api/user-state. State is also flushed on component unmount.

6. Proactive Engine

The core differentiator. The frontend hook polls POST /api/proactive every 15 seconds. The server-side engine uses **purely deterministic rules** — no Claude calls at runtime — to decide the next UI action based on the user's exploration state.

Proactive Engine — Decision Rules (Deterministic, No AI)

Rule 1	New user	→	`highlight_concept`	Pulse starting concept
Rule 2	Path exists	→	`suggest next`	Guide along exploration path
Rule 3	10-30s on node	→	`deepen_current`	Offer deeper explanation
Rule 4	Has connections	→	`suggest_connection`	Highlight unseen neighbor
Rule 5	Prereqs met	→	`show_insight`	Display insight card
Rule 6	>70% explored	→	`show_summary`	Show progress summary

All proactive decisions are deterministic — zero cost, low latency.

UI Actions

Action	UI Effect
highlight_concept	Pulse a concept node on the graph canvas
show_insight	Display a floating, color-coded insight card
suggest_connection	Highlight an edge between concepts
suggest_file	Recommend a specific file to look at
show_summary	Show exploration progress card
deepen_current	Offer deeper explanation of current concept
nothing	No change — user is engaged, don't interrupt

7. Chat System

Chat is secondary — a small input at the bottom-left of the screen, not a prominent chatbot. It has full RAG context and streams responses via SSE.

Differentiators

- **Full codebase context** — every response grounded in retrieved code chunks via hybrid search
- **Awareness of user state** — "what does this do?" knows what "this" is (selected node)
- **Adaptive language** — uses the user's estimated understanding level for the relevant concept
- **Graph integration** — [[concept:key]] and [[file:path]] references rendered as clickable links

8. End-to-End Data Flows

Runtime Data Flow

Upload Flow	Explore Flow	Chat Flow
1. User drops ZIP	1. Click concept node	1. User types question
2. Client extracts files	2. Update Zustand state	2. POST /api/chat
3. POST /api/pipeline/start	3. Sync state (5s debounce)	3. Embed question (OpenAI)
4. Server creates project	4. POST /api/explain	4. Hybrid search (70/30)
5. Client subscribes SSE	5. RAG retrieval + Claude	5. Retrieve code chunks
6. Stages 2-7 run async	6. Stream to InspectorPanel	6. Stream Claude response
7. Graph renders after S3	7. Proactive engine fires	7. Parse [[concept:]] links

Three primary user workflows: upload, explore, and chat.

9. File Structure

Server (Node.js + Hono)

Path	Purpose
server/index.ts	Hono app entry point (port 3007)
server/routes/pipeline.ts	POST /start, GET /:id/stream, GET /:id/data
server/routes/chat.ts	POST /api/chat (streaming RAG chat)
server/routes/explain.ts	POST /api/explain (streaming explanations)
server/routes/proactive.ts	POST /api/proactive (UI guidance)
server/routes/user-state.ts	PATCH /api/user-state (exploration tracking)
server/pipeline/orchestrator.ts	Pipeline orchestration (stages 1-7)
server/pipeline/fileAnalysis.ts	Stage 2: sequential file analysis (Haiku)
server/pipeline/conceptSynthesis.ts	Stage 3: concept graph generation (Sonnet)
server/pipeline/depthMapping.ts	Stage 4: multi-level explanations
server/pipeline/insightGeneration.ts	Stage 5: senior engineer insights
server/pipeline/embedding.ts	Stage 6: chunking + OpenAI embedding
server/pipeline/proactiveSeeding.ts	Stage 7: exploration path generation
server/rag/chunker.ts	~800-token chunks with contextual prefixes
server/rag/embedder.ts	OpenAI text-embedding-3-small client
server/rag/retriever.ts	Hybrid search via Supabase RPC
server/ai/claude.ts	Anthropic SDK wrapper (streaming + tool-use)
server/ai/schemas.ts	All JSON schemas for structured outputs
server/proactive/engine.ts	Deterministic proactive decision rules
server/db/supabase.ts	Supabase client

Client (React + Vite)

Path	Purpose
src/App.jsx	Screen routing (landing → upload → processing → explorer)

src/store/useStore.js	Zustand store (screens, concepts, pipeline, proactive UI)
src/hooks/useProactive.js	Polls /api/proactive every 15s
src/hooks/useSSE.js	Generic SSE stream consumption
src/hooks/useUserState.js	Track and sync user state (debounced 5s)
src/utils/fileParser.js	ZIP extraction, import detection, path resolution
src/utils/claudeApi.js	API client helpers
src/utils/graphLayout.js	D3 force-directed graph layout
src/components/GraphCanvas.jsx	D3 force-directed concept graph (Canvas)
src/components/InspectorPanel.jsx	Right sidebar: 3-level explanations, file list
src/components/ChatBar.jsx	Bottom-left chat with streaming + [[links]]
src/components/InsightCard.jsx	Floating insight cards (6 categories)
src/components/ExplorationProgress.jsx	Top banner: exploration % + suggestions
src/components/ProcessingScreen.jsx	6-step pipeline progress visualization

10. Cost Estimates

For a medium codebase (~100 files, ~50K lines):

Stage	Model	Calls	Est. Cost
Stage 2: File Analysis	claude-haiku-4-5	~7 batches of 15	~$0.10
Stage 3: Concept Synthesis	claude-sonnet-4-6	1	~$0.09
Stage 4: Depth Mapping	claude-sonnet-4-6	1	~$0.12
Stage 5: Insights	claude-sonnet-4-6	1	~$0.06
Stage 6: Embeddings	text-embedding-3-small	~1 batch	~$0.001
Stage 7: Proactive Seeding	claude-sonnet-4-6	1	~$0.03
Total Ingestion			~$0.40
Per chat message	claude-sonnet-4-6	1	~$0.015
Per explanation	claude-sonnet-4-6	1	~$0.015
Proactive engine	none (deterministic)	0	$0.00

What This Does NOT Include

- ✗ **Authentication** — Not needed for MVP. One user, one project at a time.
- ✗ **File watching** — Re-upload the ZIP. Incremental analysis adds complexity for v1.
- ✗ **Collaboration** — Single-user tool. Sharing can come later as read-only links.
- ✗ **Custom embedding fine-tuning** — Off-the-shelf embeddings with contextual retrieval are good enough.
- ✗ **Redis caching** — Supabase is the cache. Add Redis only if query latency becomes a problem.
- ✗ **Supabase Realtime** — Pipeline progress uses SSE polling from Hono. Simpler approach.